Distributed Aerial 3D Object Mapping Reconstruction Using Message Passing Interface

1st Cahyo Arissabarno
Department of Informatics and Computer Engineering
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
cahyo@pasca.student.pens.ac.id

2nd Idris Winarno*
Department of Informatics and Computer Engineering
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
idris@pens.ac.id

3rd Sritrusta Sukaridhoto
Department of Informatics and Computer Engineering
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
dhoto@pens.ac.id

2024 International Electronics Symposium (IES) | 979-8-3503-9199-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/IES63037.2024.10665858

***Abstract*—Aerial reconstruction of 3D object mapping involves utilizing drones or Unmanned Aerial Vehicles (UAVs) equipped with cameras to capture images for high-resolution 3D model reconstruction of various objects and terrains. This technique is applicable across various fields, such as Urban Planning, Vehicle Navigation, Environmental Studies, Civil Engineering, and Cultural Heritage Conservation. Drones capture hundreds of overlapping aerial photos, which are then processed to generate photorealistic 3D representations of topographic surfaces using photogrammetry software. However, the reconstruction process is time-consuming and computationally intensive, especially when processing large datasets. This research addresses these challenges by employing parallel computing using the Message Passing Interface (MPI) to distribute the computational workload across multiple nodes. By utilizing MPI, the computational tasks are efficiently divided among four nodes, significantly reducing processing time and alleviating the burden on individual CPUs. The study demonstrates that parallel computing with MPI reduces the reconstruction time from 74 minutes to 27 minutes, achieving a speedup of 2.74 times. This research demonstrates a novel application of MPI to the domain of aerial 3D reconstruction, achieving a specific and substantial reduction in processing time. Additionally, it provides insights into future enhancements, such as parallel texture reconstruction and automation of translation processes.**

Keywords—Message Passing Interface, Parallel Computing, MPI, 3D Reconstruction, Aerial 3D Object Mapping

I. Introduction

Aerial reconstruction of 3D object mapping involves utilizing drone or unmanned aerial vehicles (UAVs) equipped with cameras to capture images for high-resolution 3D model reconstruction of various objects and terrains [1]. Aerial 3D object mapping reconstruction is used to visualize a specific area in 3D model. This technique can be implemented across various fields, such as Urban Planning [2], Vehicle Navigation [3], Environmental Studies [4], Civil Engineering [5], and Cultural Heritage Conservation [6].

Capturing hundreds of overlapping aerial photos as a drone flies over a specific area enables the measurement of the area's geography and landscape features through image processing. By combining a sufficient number of overlapping images of the same features, photogrammetry software can create photorealistic 3D representations of topographic surfaces using 3D aerial reconstruction methods and software. However, this reconstruction process from aerial images to 3D objects demands a significant amount of time and computational resources, particularly when dealing with hundreds to thousands of aerial images captured by drones. The high-intensity computations over extended periods can pose risks to the CPUs used in the processing.

In response to these challenges, a promising strategy involves the adoption of parallel computing. This methodology entails the concurrent execution of applications or computations across multiple processors, thereby decomposing intricate problems into more manageable segments that can be processed concurrently. Communication between these processors typically occurs via shared memory, facilitating the aggregation of results. Parallel computing can be effectively realized through the utilization of the Message Passing Interface (MPI), a standardized communication interface widely employed in distributed computing systems. MPI serves as a robust framework for connecting multiple computing nodes across a network, enabling seamless data exchange and coordination. By leveraging MPI, parallel computing architectures can efficiently distribute computational tasks among interconnected nodes, thereby enhancing performance and scalability in tackling complex computational problems.

In this research, the Message Passing Interface (MPI) is utilized to perform parallel computing in the 3D object mapping reconstruction process. This method distributes the computational workload across four nodes, with each node handling a portion of the overall task. The results from each node are then aggregated to form a complete and coherent 3D object. By employing MPI for distributed computing, the research aims to significantly reduce the processing time and alleviate the computational burden on individual CPUs. This approach not only enhances efficiency but also minimizes the risk of overloading the processing units during intensive computational tasks. This research demonstrates a novel application of MPI in the field of aerial 3D object reconstruction, showcasing a method to significantly reduce reconstruction time and computational load.

II. Previous Works

Several previous studies have implemented the Message Passing Interface to perform parallel computing in various use cases to reduce computation time, alleviate computational load, and enhance computational scalability.

Ramadhan A *et al* [7], using MPI parallelization to speed up image stitching. Initial experiments took 1506.63 seconds, reduced to 624 seconds with MPI on two nodes, and further to 346.8 seconds with four nodes. While MPI offers faster stitching, a trade-off between speed and quality should be considered. The findings highlight MPI's potential to expedite stitching tasks, but optimization techniques may be needed for real-world applications.

*Corresponding Author

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on January 27,2026 at 13:37:12 UTC from IEEE Xplore. Restrictions apply.

Zhang J *et al* [8], develops the Reservoir GeoMechanics Simulator (RGMS) parallelized using MPI to model stresses and deformations in subsurface systems and simulate gas production from hydrate accumulations. Testing on a cluster and a Mac Pro computer demonstrates significant speedups, highlighting the effectiveness of the parallel simulators for complex simulations of large domains.

Ragunthar T *et al* [9], implements the Message Passing Interface (MPI) to parallelize the K-means clustering algorithm, enhancing its scalability and performance. By employing MPI, the efficiency of the algorithm in handling large datasets is improved. The research compares the performance of the parallel K-means algorithm with its sequential counterpart in terms of overhead cost and execution time within the MPI architecture.

Ku T *et al* [10] presents a particle-based parallel scheme for Material Point Method (MPM) using the message passing interface (MPI) to reduce computational costs. It ensures optimal load balancing by evenly distributing computational workloads across computing cores, unlike grid-based schemes. The study compares its performance with a grid-based parallel scheme.

Gamage B *et al* [11] proposes an efficient IPC architecture for Wireless Sensor Networks (WSN) using a grid-based socket approach and implements parallel AES encryption for enhanced security. Communication utilizes MPI, and encryption is parallelized with OpenMP. Results from simulations show that the proposed architecture ensures secure and efficient communication in WSNs.

Based on previous works, MPI has proven effective in accelerating processing time through parallel computing. This study uses MPI in a novel way to parallelize 3D object reconstruction from 2D aerial photos, enhancing the efficiency of aerial 3D reconstruction.

III. Proposed System

The design of the system implemented in this research has an architecture as seen in Fig. 1. In this parallel system, four nodes are utilized to distribute the 3D Object reconstruction process.

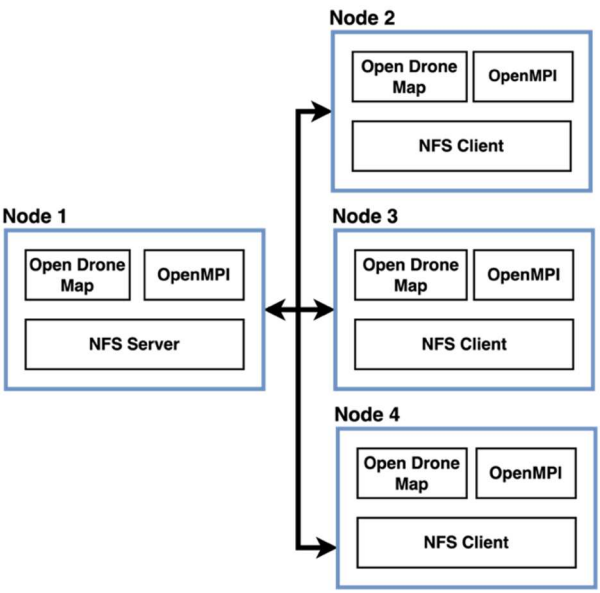

Fig. 1. Proposed System Architecture

Several crucial components of this system are Open Drone Map for the reconstruction process, OpenMPI as the Message Passing Interface manager, and NFS as the distributed file system.

A. Open Drone Map

ODM is an open-source software system for drone mapping and 3D modeling [12]. As seen in Fig. 2, the ODM workflow begins with loading the dataset, followed by creating and filtering a dense point cloud using Structure from Motion (SfM) and Multi-View Stereo (MVS). This data is then converted into a textured, georeferenced mesh. Finally, Digital Elevation Models (DEMs) and orthophotos are generated.

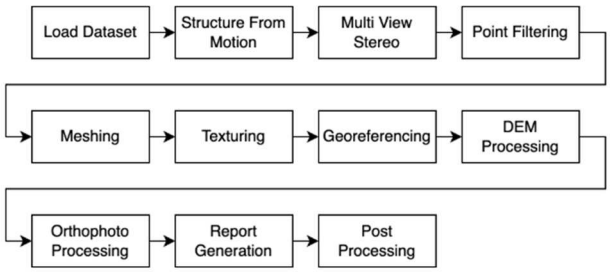

Fig. 2. ODM's processing pipeline [13]

B. OpenMPI

OpenMPI is a free implementation of the Message Passing Interface (MPI) that supports parallel and distributed computing by utilizing the capabilities of multi-processors [14]. In this research, OpenMPI is used to perform parallel computation for the reconstruction process by Open Drone Map.

C. Network File System

Network File System (NFS) is a distributed file system protocol that allows a computer to share its folders and directories with other computers over a network [7]. This enables multiple machines within a network cluster to access and use files as if they were located on their own local storage. This facilitates the MPI process in sharing datasets and integrating results from each node.

IV. Experiment and Result

This section will present the experimental environment, types of experiments and details about the devices used to represent the experimental conditions.

A. Datasets

In this experiment, an aerial photo dataset captured by a drone is utilized. As shown in Fig. 3, the flight path planning conducted to capture the dataset consists of 86 flight path points, covering an area of 332 x 205 square meters. Aerial imaging is performed with 80% photo overlapping. According to the documentation from Open DroneMap, the recommended percentage of overlap between images is between 75% and 80%.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on January 27,2026 at 13:37:12 UTC from IEEE Xplore. Restrictions apply.

Fig. 3. Flight Path

This high overlap percentage is essential for producing accurate 3D models, as it ensures multiple viewpoints of each point in the area, enhancing feature identification and image alignment during the reconstruction process.

TABLE I. CAMERA SPECIFICATION

Details	***Specification***
Camera	Hasselblad l1d-20c
Sensor size	20MP 1”
Lense	FOV 77° (28 mm) f/2.2
Type	DJI Mavic 2 Pro

As shown in Table I, aerial photos were captured using the DJI Mavic 2 Pro drone equipped with the Hasselblad L1D-20c camera. This camera is known for its 20-megapixel resolution and a large 1-inch sensor, which ensures high-quality imagery with excellent color fidelity and detail. These features are critical for accurate 3D reconstruction and photogrammetry, enabling precise mapping of terrain and objects from aerial perspectives.

TABLE II. DATA ACQUISITION DETAILS

Details	***Condition***
Date	Friday, October 07, 2022
Time	08.00 a.m.
Flight height	110 meters
Duration	± 25 minutes
Overlap	80% overlapping photo
Quantity	86 photos

This camera The DJI Mavic 2 Pro drone features a 1-inch sensor with 20-megapixel resolution, ideal for capturing detailed aerial images. Table II details specific capture conditions, including a flight height of 110 meters and 80% image overlap, optimized for accurate 3D modeling and photogrammetry. Fig. 4 provides an example image, showcasing the camera's clarity and suitability for applications like urban planning and environmental assessment.

Fig. 4. Example of Photo Captured by Drone

B. 3D Reconstruction Methods

Open Drone Map has provided various methods for reconstructing 3D object mapping. For feature extraction, there are methods such as SIFT, AKAZE, HAHOG, and ORB. For feature matching, there are methods like BOW, BruteForce, and FLANN. Both types of parameters can be combined to achieve reconstruction results according to preferences. In this research, the feature extraction method used is HAHOG, and the feature matching method used is BruteForce. This is based on a study by Satriawan J et al [15], which found that the combination of HAHOG and BruteForce yields high accuracy but has the longest processing time among other combinations.

C. MPI

To implement parallel computing, this research utilizes four virtual machine nodes, each with specifications as shown in Table III. The virtual machines used are hosted on the campus server.

TABLE III. NODE SPECIFICATION

Items	***Specification***
Processor	Intel(R) Xeon(R) CPU E5-2650 v2 @2.30 GHz
RAM	16 GB
Storage	20 GB

To perform distributed computing processes, one of the open-source MPI software, namely OpenMPI version 3.1.3, is utilized. Processing is carried out by distributing the 86 dataset images to four nodes to be processed using Open Drone Map on each node. Once the process on each node is completed, the results will be stored in a directory configured as NFS. The results from all nodes will be merged by node 1.

Fig. 5. 3D Reconstruction Time Without MPI

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on January 27,2026 at 13:37:12 UTC from IEEE Xplore. Restrictions apply.

The first experiment was conducted without MPI. This experiment was performed using the Web Interface of OpenDroneMap. From this experiment, the processing time obtained was 1 hour 14 minutes 50 seconds, or approximately 74.83 minutes when rounded. This can be seen in Fig. 4.

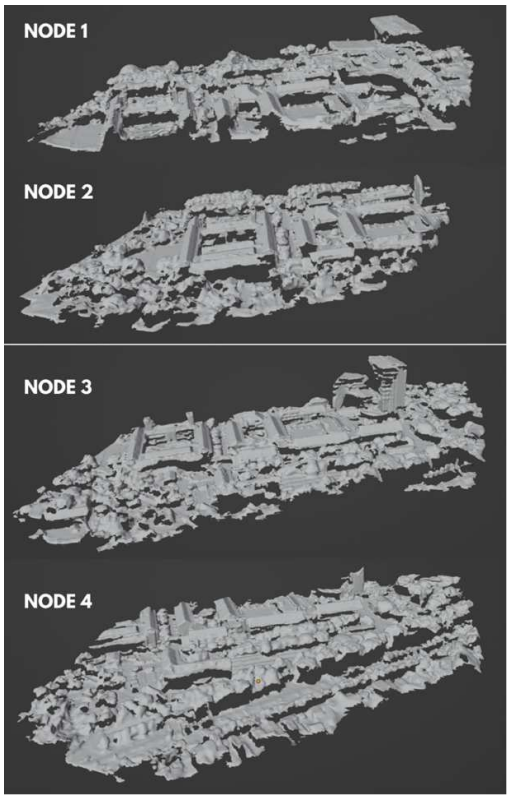

Fig. 6. 3D Result From Each Node

The second experiment was conducted using MPI with 4 nodes. This processing utilizes the Python library from OpenDroneMap, namely PyODM, which accesses the ODM software running on each node. The 3D results from each node, as seen in Fig. 5, will be combined using the trimesh library in Python. All of these libraries are integrated with an MPI program created using the MPI4PY library.

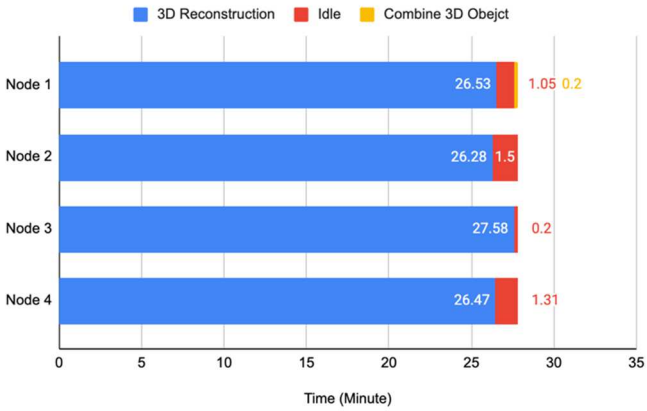

Fig. 7. 3D Reconstruction Time Using MPI

As seen in Fig. 6, the time required to perform the 3D reconstruction process on each node is relatively similar, around 26 - 27 minutes. Thus, the average time for each node is 26.71 minutes. The Idle time indicates the condition of a node that is still in standby mode but has completed the 3D reconstruction process on that node.

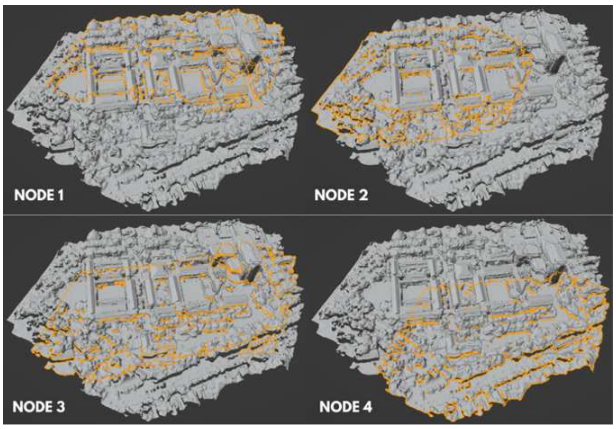

Fig. 8. 3D Position From Each Node in Combine Process

The merging process is conducted by node 1 and takes approximately 0.2 minutes or about 12 seconds. This merging procedure involves performing a translation process to integrate the individual 3D reconstructions into a comprehensive final model, as illustrated in Fig. 7.

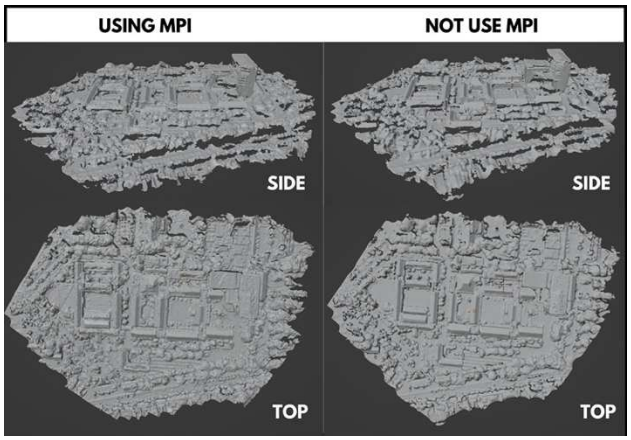

Fig. 9. 3D Result Comparison

In this study, the translation calculation process is still manually defined within the code. From the experiment results with MPI and without MPI, the 3D reconstruction outcomes can be observed as depicted in Fig. 8. It can be seen that there is no significant difference between the two types of experiments, and it is evident that parallel processing with 4 nodes accelerates the process by 2.74 times compared to processing without MPI.

V. Conclusion

In this research, the distributed reconstruction process of 3D object maps was undertaken through parallel computing utilizing the Message Passing Interface method. OpenMPI served as the software for Message Passing Interface operations, while OpenDroneMap was employed for the actual 3D reconstruction. The experiments conducted revealed that parallel computation for the 3D reconstruction process took approximately 27 minutes, whereas performing the same task without MPI consumed about 74 minutes.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on January 27,2026 at 13:37:12 UTC from IEEE Xplore. Restrictions apply.

Thus, employing MPI with 4 nodes accelerated the process by a factor of 2.74. Several limitations were identified in this study, including the inability to conduct texture reconstruction in parallel and the reliance on manual translation procedures. These findings open avenues for future research endeavors in the domain of parallel computing and 3D reconstruction.

REFERENCES

[1] M. Moreno-Barahona, B. Fraijo-Sing, G. Fleury-Bahi, O. Navarro-Carrascal, and C. Tapia-Fonllem, "Conceptual Integration and Empirical Validation of a Unified Taxonomy: Quantitative Data Analysis for Virtual Learning Environments," *Front Psychol*, vol. 13, 2022, doi: 10.3389/fpsyg.2022.814592.

[2] F. Yan, E. Xia, Z. Li, and Z. Zhou, "Sampling-based path planning for high-quality aerial 3D reconstruction of urban scenes," *Remote Sens (Basel)*, vol. 13, no. 5, pp. 1–23, Mar. 2021, doi: 10.3390/rs13050989.

[3] H. Zhang, C. Xie, H. Toriya, H. Shishido, and I. Kitahara, "Vehicle Localization in a Completed City-Scale 3D Scene Using Aerial Images and an On-Board Stereo Camera," *Remote Sens (Basel)*, vol. 15, no. 15, Aug. 2023, doi: 10.3390/rs15153871.

[4] Y. Qiu *et al.*, "A Rapid Water Region Reconstruction Scheme in 3D Watershed Scene Generated by UAV Oblique Photography," *Remote Sens (Basel)*, vol. 15, no. 5, Mar. 2023, doi: 10.3390/rs15051211.

[5] H. Li, Y. Chen, J. Liu, C. Che, Z. Meng, and H. Zhu, "High-resolution model reconstruction and bridge damage detection based on data fusion of unmanned aerial vehicles light detection and ranging data imagery," *Computer-Aided Civil and Infrastructure Engineering*, vol. 39, no. 8, pp. 1197–1217, Apr. 2024, doi: 10.1111/mice.13133.

[6] Q. Li *et al.*, "Single drone-based 3D reconstruction approach to improve public engagement in conservation of heritage buildings: A case of Hakka Tulou," *Journal of Building Engineering*, vol. 87, p. 108954, 2024, doi: https://doi.org/10.1016/j.jobe.2024.108954.

[7] A. W. Ramadhan, F. Aulia, N. M. L. A. Dewi, I. Winarno, and S. Sukaridhoto, "Distributed Aerial Image Stitching on Multiple Processors using Message Passing Interface," *JOIV: International Journal on Informatics Visualization*, vol. 8, no. 1, pp. 409–416, 2024.

[8] J. Zhang, G. Moridis, and T. Blasingame, "Message-Passing-Interface MPI Parallelization of Iteratively Coupled Fluid Flow and Geomechanics Codes for the Simulation of System Behavior in Hydrate-Bearing Geologic Media," in *SPE Annual Technical Conference and Exhibition*, Dubai, UAE, Sep. 2021. doi: https://doi.org/10.2118/206161-MS.

[9] T. Ragunthar, P. Ashok, N. Gopinath, and M. Subashini, "A strong reinforcement parallel implementation of k-means algorithm using message passing interface," *Mater Today Proc*, vol. 46, pp. 3799–3802, 2021, doi: https://doi.org/10.1016/j.matpr.2021.02.032.

[10] T.-H. Ku and H.-G. Kim, "A particle-based parallel scheme for material point method (MPM) using message passing interface (MPI)," *Comput Part Mech*, vol. 10, no. 1, pp. 61–76, 2023, doi: 10.1007/s40571-022-00480-y.

[11] B. M. S. V. Gamage and V. M. Baskaran, "Simulation and Analysis of Distributed Wireless Sensor Network using Message Passing Interface," *CoRR*, vol. abs/2007.00754, 2020, [Online]. Available: https://arxiv.org/abs/2007.00754

[12] A.-M. G. Gbagir, K. Ek, and A. Colpaert, "OpenDroneMap: Multi-Platform Performance Analysis," *Geographies*, vol. 3, no. 3, pp. 446–458, 2023, doi: 10.3390/geographies3030023.

[13] Piero Toffanin, *OpenDroneMap: The Missing Guide*, Second Edition. UAV4GEO, 2023.

[14] J. C. Adams, "OpenMPI+ Java as a High Performance Language," in *2022 IEEE/ACM International Workshop on Education for High Performance Computing (EduHPC)*, 2022, pp. 11–17. doi: 10.1109/EduHPC56719.2022.00007.

[15] J. Satriawan, I. Winarno, and D. Pramadihanto, "Comparative Reconstruction 3D Object Mapping Methods Using Drone," in *IES 2023 - International Electronics Symposium: Unlocking the Potential of Immersive Technology to Live a Better Life, Proceeding*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 281–285. doi: 10.1109/IES59143.2023.10242596.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on January 27,2026 at 13:37:12 UTC from IEEE Xplore. Restrictions apply.